

A COOKBOOK FOR ENGINEERS

MLOps Cookbook

Ship and operate machine learning in production — experiment tracking, versioning, pipelines, serving, monitoring, and CI/CD for models — the engineering half of the ML engineer role.

MLflow

Model registry

CI/CD

Serving

Monitoring

Drift

Pipelines

Feature stores

Written for a developer who can build models (see the ML/DL books) and now needs to run them in production like real software.

Every chapter answers: what it solves, how to do it, the failure modes, and when a simpler setup wins.

Edition 2026 · a model is a service · reproducible, monitored, shippable

Table of Contents

How to Use This Book	4
The one idea: reproducibility → automation → monitoring.	4
What this book covers.	5
Part 1 · What MLOps Is (and the Maturity Ladder)	6
1.1 Why ML needs more than DevOps.	6
1.2 The MLOps maturity ladder	6
1.3 What lives in each layer	7
Part 2 · Experiment Tracking	8
2.1 What to track, every run.	8
2.2 MLflow in practice	8
2.3 Why it's the highest-leverage step	8
Part 3 · Versioning Data & Models	10
3.1 Why Git alone isn't enough	10
3.2 Data versioning with DVC	10
3.3 Model versioning	11
3.4 The reproducibility contract	11
Part 4 · Training Pipelines	12
4.1 From notebook to pipeline.	12
4.2 The tools.	12
4.3 Make steps idempotent and parameterised	12
Part 5 · The Model Registry	14
5.1 What a registry gives you	14
5.2 With MLflow	14
5.3 Decoupling train from deploy	15
Part 6 · Serving & Deployment	16
6.1 Online vs batch — pick the right shape	16
6.2 The online serving pattern.	16
6.3 Safe rollouts	17
6.4 Training/serving skew — the classic bug.	17
Part 7 · Monitoring & Drift	18
7.1 Two layers of monitoring.	18
7.2 The three things that drift	18
7.3 How to monitor quality	18
7.4 The ground-truth delay	19
Part 8 · CI/CD for Machine Learning	20
8.1 Continuous Integration — test more than code	20
8.2 Continuous Training (CT) — the ML-specific loop	20
8.3 The quality gate is the safety mechanism.	21
Part 9 · Feature Stores (Advanced)	22
9.1 The problem it solves	22

9.2 Point-in-time correctness	22
9.3 Be honest: you probably don't need one yet	23
Part 10 · Capstone & Cheat Sheet	24
10.1 Capstone — take a model from notebook to operated service.	24
10.2 The MLOps loop at a glance	24
10.3 Tooling cheat sheet.	24
10.4 The five rules.	25

How to Use This Book

You can train a model now — the ML, Deep Learning, and Fine-Tuning books got you there. But a trained model in a notebook is worth nothing to a business. **MLOps** is the discipline that turns "I trained a model" into "we run a model in production that people depend on": reproducibly, observably, and updatable without a heroics-driven redeploy. This is the engineering half of the ML engineer role, and it's where your software background is a genuine advantage.

The core reframe: **a model is a service with a data supply chain**. It has inputs, outputs, dependencies, versions, tests, a deploy, and — uniquely — it can silently get *worse* over time as the world drifts from its training data. MLOps is DevOps plus the extra discipline that last fact demands.

Each chapter answers four questions:

1. **What problem does this solve?**
2. **How do you do it (in Python / tooling)?**
3. **What are the failure modes?**
4. **When is a simpler setup the right call?**

The one idea: reproducibility → automation → monitoring

Three layers, built in order:

DIAGRAM

REPRODUCIBILITY (same inputs, same model) track & version	→	AUTOMATION (pipelines + CI/CD retrain) no manual steps	→	MONITORING (catch drift & decay in prod) a model rots silently
--	---	---	---	---

You can't automate what you can't reproduce, and you can't trust what you don't monitor. Most MLOps failures are skipping straight to a fancy serving stack without the reproducibility underneath.

ENGINEER'S MENTAL MODEL

MLOps is DevOps for models, with one extra hazard. Your CI/CD, containers, versioning, and monitoring instincts all transfer directly. The new part is that a deployed model degrades *without any code change* — the data feeding it shifts, and yesterday's accurate model is today's liability. Everything unique about MLOps (versioning *data*, monitoring *drift*, scheduled *retraining*) exists to handle that one fact.

What this book covers

The MLOps mindset and maturity ladder; **experiment tracking** (MLflow); **data and model versioning**; **training pipelines**; the **model registry** and promotion; **serving/deployment** patterns; **monitoring and drift**; **CI/CD for ML** (including retraining); and **feature stores**. It's tool-aware but principle-first — tools change, the problems don't.

HOW TO READ IT

Read Parts 1-3 in order — the mindset, tracking, and versioning are the foundation everything else assumes. Then Parts 4-8 are the production pipeline (pipelines → registry → serving → monitoring → CI/CD), roughly the order you'd build them. Part 9 (feature stores) is advanced/optional. Do the exercises against a real model you trained in the other books — MLOps only makes sense wrapped around something real.

READ THIS FIRST: MATCH THE RIGOR TO THE STAKES

Full MLOps is for models that **matter** — real users, real money, frequent updates. A one-off analysis, a proof-of-concept, or a model that runs once a quarter does **not** need a feature store, a registry, and automated retraining; that machinery is cost you don't need yet. Start with the minimum (track experiments, version the model, serve it, watch it) and add layers only as the model's importance grows. Over-engineered MLOps is as real a failure as none.

Part 1 · What MLOps Is (and the Maturity Ladder)

MLOps is the set of practices for deploying and *maintaining* ML models in production reliably. It's not one tool — it's a discipline, and teams sit at very different maturity levels. Knowing where you are (and the next rung, not the top) is how you avoid over-building.

1.1 Why ML needs more than DevOps

A traditional service is defined by its code. An ML system is defined by **code + data + model**, and that changes everything:

Concern	Traditional software	ML system
What defines it	code	code + data + model weights
"It works"	tests pass	tests pass + metrics hold on live data
Fails by	a bug you shipped	the world drifting from training data
To reproduce	the commit	commit + data version + config + seed
Deploy	ship the binary	ship model + keep watching it

The extra dimensions — data and a model that decays — are why "just use DevOps" isn't enough.

ENGINEER'S MENTAL MODEL

Think of the model as a *compiled artifact* whose source is your dataset and training config. To reproduce a build you need the exact source (data version), the compiler settings (hyperparameters, seed), and the toolchain (library versions). Miss any and you can't rebuild the same model — which is why "it worked on my machine" is even worse in ML than in normal software.

1.2 The MLOps maturity ladder

Most teams climb this; find your rung and target the next one:

- **Level 0 — Manual.** Notebooks, models trained and deployed by hand, no tracking. Fine for a first model; unsustainable past one.
- **Level 1 — Reproducible.** Experiments tracked, data and models versioned, training is a script not a notebook. *This is the highest-leverage jump.*
- **Level 2 — Automated pipelines.** Training and deployment are pipelines you can re-run on new data with one command.

- **Level 3 — Continuous.** Monitoring triggers retraining; CI/CD tests and promotes models automatically; drift is caught and handled.

1.3 What lives in each layer

DIAGRAM

```

track experiments → version data+model → pipeline → registry → serve → monitor → retrain
└── reproducibility ─┘      └── automation ─┘      └── operation & feedback ─┘
  
```

The rest of the book walks this left to right. Crucially, **build it left to right** — a serving stack (right) on top of un-versioned, untracked models (left) is a liability, not progress.

COMMON MISTAKE

Buying/adopting a big MLOps platform before you've mastered Level 1. Teams stand up an elaborate serving and orchestration stack while their models are still trained in ad-hoc notebooks with no tracking or versioning — so they *still* can't answer "which data made the model in prod?" The foundation (reproducibility) delivers more value than any serving tool; earn each rung before reaching for the next.

WHEN NOT TO CLIMB HIGHER

Don't chase Level 3 for a model that's retrained twice a year, or a proof-of-concept that may never ship. The automation and continuous-retraining machinery is justified by **change frequency and stakes** — a stable, low-risk model is fine at Level 1. The right level is the lowest one that meets the model's real operational demands.

EXERCISE 1.1

Rate a model you've built (or a hypothetical production one) on the 0–3 ladder, honestly. Then write down the single next rung and the *one* practice that gets you there (usually: start tracking experiments and versioning the model). Note what value that one step adds — and resist listing five levels of tooling you don't yet need.

Part 2 · Experiment Tracking

The first real MLOps practice, and the highest-leverage one: record every training run's inputs and outputs so you can answer "what did I change, and did it help?" Without tracking, ML becomes a fog of forgotten notebooks and un-reproducible numbers. **MLflow** is the common tool; the idea matters more than the tool.

2.1 What to track, every run

- **Parameters** — hyperparameters, model type, data version.
- **Metrics** — accuracy/loss/F1 on train and validation, per run (and per epoch).
- **Artifacts** — the trained model file, plots, the config.
- **Code version** — the git commit that produced the run.
- **Environment** — library versions (so the run is reproducible).

2.2 MLflow in practice

PYTHON

```
import mlflow

mlflow.set_experiment("churn-classifier")
with mlflow.start_run():
    mlflow.log_params({"model": "xgboost", "max_depth": 6, "data_version": "v3"})
    model = train(...)
    mlflow.log_metrics({"val_auc": 0.91, "val_f1": 0.78})
    mlflow.sklearn.log_model(model, "model")      # stores the artifact
    mlflow.set_tag("git_sha", current_commit())
```

Now every run is a row you can search, sort, and compare in the MLflow UI — which hyperparameters gave which metrics, with the exact model attached.

ENGINEER'S MENTAL MODEL

Experiment tracking is structured logging for training runs — but instead of debugging one execution, you're building a searchable history to compare hundreds. It's your lab notebook as a database: "which run had the best val AUC, and what data + params produced it?" becomes a query, not an archaeology dig through notebook cells you've since overwritten.

2.3 Why it's the highest-leverage step

ML is relentlessly iterative — you'll run dozens of variants. Without tracking you *will* lose the good one ("wait, which settings got 0.91?"), you can't prove a change helped, and you

can't reproduce last month's model. Tracking turns that chaos into a comparable, reproducible record for near-zero effort.

COMMON MISTAKE

Tuning in notebooks and recording results in your head, a scratch cell, or a spreadsheet you forget to update. Two days later you can't recreate the run that worked, can't tell if today's tweak actually beat it, and are comparing numbers computed on different data splits. Wrap training in a tracked run from the *first* experiment — retrofitting tracking after you've lost the good run is too late.

PRODUCTION TIP

Log the **data version** and **git SHA** on every run, not just hyperparameters. The most painful reproducibility failure isn't a forgotten learning rate — it's not knowing *which data* trained the model, because the dataset changed under you between runs. With the data version and commit recorded, any run is fully reconstructable; without them, "best" runs become unrepeatable folklore.

EXERCISE 2.1

Wrap a model you trained (ML/DL book) in MLflow tracking: log params, val metrics, the model artifact, the data version, and the git SHA. Run it 5 times with different hyperparameters, then use the MLflow UI to find the best run and confirm you can identify exactly what produced it. Delete your notebook's in-memory state and prove you can still reconstruct that run.

Part 3 · Versioning Data & Models

You version code with Git. In ML you must *also* version the **data** and the **model**, because those — not just code — determine behaviour. Without it, "which data produced the model in production?" is unanswerable, and reproducibility is a fiction. This is the other half of the reproducibility foundation.

3.1 Why Git alone isn't enough

Git is built for text, not gigabyte datasets or model binaries. Committing them bloats the repo and doesn't capture the *link* between a data version, a training run, and the resulting model. You need tools that version large files and tie the three together.

Artifact	Versioned with
Code	Git
Data	DVC, LakeFS, or dataset snapshots (versioned storage)
Model	MLflow registry (Part 5), or DVC / object storage with tags
The <i>link</i> between them	your tracking (Part 2): run → data ver + commit + model

3.2 Data versioning with DVC

DVC ("Git for data") stores large files in remote storage and keeps small pointer files in Git, so a `git checkout` of an old commit brings back the exact data that went with it:

SHELL

```
dvc init
dvc add data/train.parquet      # DVC tracks the file; Git tracks a tiny .dvc
                                # pointer
git add data/train.parquet.dvc .gitignore && git commit -m "data v3"
dvc push                        # upload the actual data to remote storage (S3/GCS)
# later, on another machine or an old commit:
git checkout <sha> && dvc pull  # get the exact data for that commit
```

ENGINEER'S MENTAL MODEL

DVC is Git-LFS with a brain: Git holds a pointer, the real bytes live in object storage, and checking out a commit restores the matching data. The goal is that **any past model is fully reconstructable** — `git checkout` a commit, `dvc pull` its data, re-run training, get the same model. That's a reproducible build for ML, and it's impossible if data floats around unversioned.

3.3 Model versioning

Every trained model needs an identity: a version, the data + config that made it, and its eval scores. Lightweight approaches tag model files in object storage with that metadata; the cleaner approach is a **model registry** (Part 5) that makes versions, lineage, and promotion first-class.

3.4 The reproducibility contract

A model is reproducible only if you can recover *all* of: the **code** (commit), the **data** (version), the **config** (hyperparameters + seed), and the **environment** (pinned libraries). Tracking (Part 2) records them; versioning makes the data and model recoverable. Together they're the contract.

COMMON MISTAKE

Overwriting datasets in place — `train.csv` gets "updated" and the old version is gone forever. Now every model trained on the old data is irreproducible, and you can't tell whether a metric change came from your code or from silently different data. Treat datasets as **immutable, versioned snapshots** (append a new version, never overwrite), exactly as you'd never force-push over main.

WHEN NOT TO ADD HEAVY DATA VERSIONING

For a small static dataset that never changes, a checked-in file (or a hash in your tracking) is enough — you don't need DVC and a remote store. Data versioning earns its keep when the data is **large, changing, or shared across a team**. Match the tool to the data's actual volatility and size.

EXERCISE 3.1

Put a dataset under DVC with a remote (a local folder is fine as the "remote"). Train a model, commit code + the `.dvc` pointer + the model's tracked run. Change the data, commit "v2", retrain. Then `git checkout` the first commit, `dvc pull`, retrain, and confirm you get the original model back — the reproducibility contract, proven.

Part 4 · Training Pipelines

A model trained by running notebook cells in the right order by hand is not reproducible or automatable. A **pipeline** turns training into a defined sequence of steps — data prep → train → evaluate → register — that anyone (or a schedule) can run with one command and get the same result. This is the jump from Level 1 to Level 2.

4.1 From notebook to pipeline

The steps are the same; the difference is they become **parameterised, ordered, re-runnable code** instead of manual cell execution:

DIAGRAM

```
[ingest+validate data] → [preprocess/features] → [train] → [evaluate] → [register if better]
each step: typed inputs/outputs, cached, re-runnable, logged
```

Even a plain Python script with functions and a `main()` beats a notebook. Real pipeline tools add caching, parallelism, retries, and a run history on top.

4.2 The tools

- **A structured script / Makefile** — the honest starting point; version it, run it with a config. Don't skip past this.
- **DVC pipelines** — define stages and dependencies in `dvc.yaml`; re-runs only what changed.
- **Airflow / Dagster / Prefect** — general orchestrators for scheduled, monitored DAGs (also core Data Engineering tools).
- **Kubeflow / SageMaker Pipelines / Vertex Pipelines** — ML-specific, Kubernetes/cloud-native.

ENGINEER'S MENTAL MODEL

A pipeline is a build system for models: stages with declared inputs and outputs, so the tool can cache unchanged steps and re-run only what's stale — `make` for ML. The payoff is the same as CI: a green, repeatable, one-command build removes "did I run the cells in the right order?" as a source of bugs and makes automated retraining (Part 8) possible at all.

4.3 Make steps idempotent and parameterised

Each stage should take its inputs explicitly (data version, config) and produce the same output given the same inputs — no hidden global state, no "run this cell first." That's what lets a schedule or CI run it unattended and trust the result.

COMMON MISTAKE

Shipping the notebook *as* the pipeline — "just run these cells top to bottom." Notebooks invite hidden out-of-order state, manual edits between runs, and un-parameterised paths, so the "same" run gives different results on a different day or machine. Extract the logic into ordered, parameterised functions/scripts; keep notebooks for exploration, not for the production training path.

WHEN NOT TO REACH FOR A HEAVY ORCHESTRATOR

A single model retrained occasionally does **not** need Airflow or Kubeflow — a versioned script run by cron (or a DVC pipeline) is simpler, cheaper, and easier to debug. Adopt a full orchestrator when you have **many** interdependent pipelines, real scheduling/monitoring needs, or a team sharing them. Starting with Airflow for one model is over-engineering.

EXERCISE 4.1

Convert a training notebook into a parameterised pipeline: separate stages for data prep, train, and evaluate, each a function taking explicit inputs and writing explicit outputs, driven by a config file and wrapped in MLflow tracking. Run it end to end with one command, then re-run with a different config and confirm you get a cleanly tracked, reproducible second run.

Part 5 · The Model Registry

Once you're producing many model versions, you need one authoritative place that answers "which model is in production, which is staging, and how did it get there?" That's the **model registry** — the source of truth for model versions, their lineage, and their promotion through stages. It's what decouples *training* a model from *deploying* one.

5.1 What a registry gives you

- **Named, versioned models** — `churn-classifier v7`, not `model_final_v2.pkl`.
- **Stages** — each version sits in a stage: `Staging`, `Production`, `Archived`.
- **Lineage** — links back to the run, data version, and metrics that produced it (Part 2).
- **Promotion & rollback** — move a version to Production, or roll back to the previous one, without retraining or redeploying code.

DIAGRAM

```
train run → register as v7 → Staging → (tests/eval pass) → Production
                                                    | rollback ↶
                                                    v6 still one click away
```

5.2 With MLflow

PYTHON

```
import mlflow
# register the model produced by a run
mlflow.register_model("runs:/<run_id>/model", "churn-classifier")
# promote a version to Production (serving reads whatever is in this stage)
from mlflow import MlflowClient
MlflowClient().transition_model_version_stage("churn-classifier", version=7,
stage="Production")
```

Your serving layer (Part 6) loads "the Production version of churn-classifier" rather than a hard-coded file — so promoting or rolling back is a registry change, not a redeploy.

ENGINEER'S MENTAL MODEL

The registry is a container registry (or artifact repo) for models: immutable versioned artifacts, environment tags (staging/prod), and promotion between them. "Deploy" becomes "point the prod tag at v7," and "roll back" becomes "point it back at v6" — the same decoupling of build-artifact from running-environment you already rely on for services, applied to models.

5.3 Decoupling train from deploy

The key win: training produces a *candidate*; a separate, deliberate promotion step decides what serves. That gap is where your quality gate lives — automated eval, a human approval, a canary — before a new model reaches users (Part 8).

COMMON MISTAKE

Serving a model from a hard-coded file path (`/models/model.pkl`) that people overwrite to "deploy." There's no version history, no lineage, no clean rollback, and no record of what's actually live — and two people overwriting it race each other. Put models in a registry and have serving load *by name + stage*, so deploys and rollbacks are tracked, atomic, and reversible.

WHEN NOT TO RUN A FULL REGISTRY

For a single model deployed rarely, a versioned object-storage bucket with clear tags (and lineage in your tracking) can be enough — a full registry is overhead you may not need yet. The registry earns its place once you have **multiple models, multiple versions in flight, or a team** promoting and rolling back regularly.

EXERCISE 5.1

Register a trained model in the MLflow registry, promote v1 to Production, then train an improved v2, register it, and promote it — confirming the old version is retained and you can transition back to it in one call. Make a small script that loads "the Production version" by name (not a file path) so a promotion changes what it serves with no code edit.

Part 6 · Serving & Deployment

Serving is turning a registered model into predictions other systems can call. The mechanics are familiar backend work — an endpoint, a container, a load balancer — but ML adds a few patterns worth knowing: online vs batch, the safe rollout strategies, and why the *preprocessing* is the part that bites you.

6.1 Online vs batch — pick the right shape

	Online (real-time)	Batch
Shape	an API returns a prediction per request	score many rows on a schedule
Use when	you need a prediction <i>now</i> (fraud, recommend)	predictions can be precomputed (nightly scores)
Cost/complexity	higher (latency SLAs, always-on)	lower (a scheduled job)

Many problems that people build a real-time API for are actually **batch** — if a prediction can be precomputed and looked up, batch is far cheaper and simpler.

6.2 The online serving pattern

Load the Production model from the registry and wrap it in an API (FastAPI, your Python book) in a container:

PYTHON

```
from fastapi import FastAPI
import mlflow.pyfunc

app = FastAPI()
model = mlflow.pyfunc.load_model("models:/churn-classifier/Production") # by
name+stage

@app.post("/predict")
def predict(features: dict):
    return {"prediction": model.predict([features]).tolist()}
```

Containerize it (Docker, your Docker book), put it behind autoscaling (Kubernetes, the Cloud & Kubernetes book), and you have a scalable model service. For heavy load, a purpose-built server (vLLM for LLMs, Triton/TorchServe for others) batches requests for you.

ENGINEER'S MENTAL MODEL

Serving a model is serving any stateless service — your entire backend skillset applies: containers, health checks, autoscaling, load balancing, timeouts. The only ML-specific twists are (a) the artifact is a model loaded from a registry, and (b) the input must be preprocessed *identically* to training (6.4). Don't over-mystify it; it's a FastAPI service with an unusual dependency.

6.3 Safe rollouts

A new model can be worse in ways tests miss. Roll out like risky code:

- **Shadow** — send real traffic to the new model, compare to the old, but don't act on its output yet.
- **Canary** — route a small % of traffic to the new model, watch metrics, ramp up.
- **A/B** — split traffic to measure the new model's real impact.
- **Rollback** — one registry transition back to the previous version (Part 5).

6.4 Training/serving skew — the classic bug

The features at serving time must be computed **exactly** as at training time. Different code paths, a different library version, a subtly different normalization, or a feature that's available in training but not live — any of these makes the model see inputs it wasn't trained on, and accuracy quietly collapses.

COMMON MISTAKE

Training/serving skew: the training pipeline computes a feature one way (in pandas, over the whole dataset) and the serving code computes it another way (per request, slightly differently) — or uses a stat that leaked from the future. The model degrades in production with no error and passing unit tests. Share the *same* preprocessing code between train and serve, and validate live inputs against the training schema.

WHEN NOT TO BUILD A REAL-TIME SERVICE

If predictions don't need to be instant, don't stand up an always-on, autoscaled API — run a **batch** job that scores rows on a schedule and writes results to a table your app reads. It's cheaper, simpler, and has no latency SLA to defend. Reserve online serving for genuinely real-time decisions.

EXERCISE 6.1

Serve a registered model behind a FastAPI `/predict` endpoint that loads the Production version *by name+stage*, and containerize it. Then implement the same model as a batch job that scores a table on a schedule. Finally, introduce a deliberate training/serving skew (normalize differently at serve time) and observe predictions degrade — then fix it by sharing the preprocessing code.

Part 7 · Monitoring & Drift

This is the chapter with no equivalent in normal software, and the one that justifies MLOps existing. A deployed model **decays** — not because the code broke, but because the world moved away from its training data. If you're not watching for that, you're running a model that's silently getting worse and nobody knows.

7.1 Two layers of monitoring

- **Operational** (same as any service) — latency, throughput, error rate, uptime, resource use. Your existing observability (the infra/observability material).
- **Model-quality** (ML-specific) — is it still *accurate*, and does its input still look like training data? This is the new part.

7.2 The three things that drift

- **Data drift** — the input distribution shifts (a feature's range, mix, or meaning changes; a new customer segment; a sensor recalibrated).
- **Concept drift** — the *relationship* between inputs and the target changes (what predicted churn last year no longer does). The model can't fix this itself.
- **Prediction drift** — the model's output distribution shifts (suddenly predicting "fraud" 5× as often) — often the first symptom you can see.

DIAGRAM

training world $\neq$ today's world $\rightarrow$ accuracy quietly falls
 watch: input distributions, prediction distribution, and (when labels arrive) real accuracy

ENGINEER'S MENTAL MODEL

Drift is the model's dependencies changing without a version bump — like an upstream API subtly altering its response while your code sits unchanged and starts producing garbage. There's no exception, no failed test; only monitoring catches it. That's why "deploy and forget" is safe for a function and dangerous for a model.

7.3 How to monitor quality

- **Track input feature distributions** vs a training baseline; alert on significant shifts (statistical distance tests; tools like Evidently, or Prometheus/Grafana on your own metrics).
- **Track the prediction distribution** — cheap, needs no labels, catches a lot.

- **Track real accuracy when labels arrive** — often *delayed* (you learn if a loan defaulted months later). Log predictions so you can join them to outcomes when the truth shows up.
- **Alert and trigger retraining** when metrics cross a threshold (Part 8).

7.4 The ground-truth delay

The hardest part: you usually don't know if a prediction was right *at the time*. Design for it — **log every prediction with its inputs and an id**, so that when the real outcome arrives later you can join them and compute true accuracy retrospectively. Without that logging, you can never measure how good the live model actually was.

COMMON MISTAKE

Monitoring only uptime and latency — treating the model like a normal service — and declaring it healthy while its accuracy has quietly halved. A model can be 100% "up," fast, and error-free while being badly wrong, because drift throws no exception. Monitor **input and prediction distributions and (delayed) accuracy**, not just that the endpoint responds.

PRODUCTION TIP

Log **every prediction with its input features, model version, and a request id**, to durable storage. It's the backbone of everything: joining to delayed labels for real accuracy, computing drift, debugging "why did it predict that?", and building the next training set from real production data. A model service that doesn't log its predictions is flying blind.

EXERCISE 7.1

For a served model, log every prediction with inputs, model version, and id. Compute and store the training feature distribution as a baseline, then feed the service deliberately shifted inputs and detect the data drift (a simple per-feature distance vs baseline). Simulate delayed labels arriving and join them to logged predictions to compute real accuracy after the fact.

Part 8 · CI/CD for Machine Learning

CI/CD for ML is your normal pipeline plus two ML-specific ideas: tests that check *data and model quality* (not just code), and **continuous training** — retraining triggered by a schedule or by drift, with an automatic quality gate before anything reaches users. This is the top of the maturity ladder.

8.1 Continuous Integration — test more than code

An ML change (new data, new features, new model) needs tests beyond unit tests:

- **Code tests** — the usual (preprocessing functions, the serving API).
- **Data tests** — schema, ranges, null rates, distribution sanity on incoming data (catch bad data *before* it trains a bad model).
- **Model tests** — does the newly trained model clear a metric bar on the held-out set, and beat the current Production model? Does it pass behavioural checks (no regression on key segments)?

DIAGRAM

```
PR / new data → CI: code tests + data validation + train + eval-vs-prod
                  └─ promote to Staging only if it clears the gate
```

8.2 Continuous Training (CT) — the ML-specific loop

Because models decay (Part 7), the pipeline doesn't just run on code changes — it runs on **new data or on drift**:

DIAGRAM

```
trigger (schedule OR drift alert) → pipeline retrains → eval vs Production
    |                                     |
    └─ promote & deploy only if it's better ← else keep current
```

The non-negotiable rule: **a retrained model is a candidate, not a deployment**. It must beat the incumbent on your eval gate before promotion — automated retraining without an eval gate is a way to auto-deploy a *worse* model.

ENGINEER'S MENTAL MODEL

CI/CD for ML is your existing pipeline with data and model quality as first-class test stages, plus a trigger that isn't only "someone pushed code" — it's "new data arrived" or "monitoring saw drift." The deploy step gains a quality gate (beat prod on held-out metrics) exactly like a performance regression check gates a normal release. Same machinery, ML-shaped gates.

8.3 The quality gate is the safety mechanism

Every promotion — manual or automated — passes through: train → evaluate on a fixed held-out set → compare to Production → (optional canary, Part 6) → promote or reject. This gate is what makes automation safe. Skip it and CT becomes auto-shipping unreviewed models.

COMMON MISTAKE

Automating retraining without an evaluation gate — a schedule retrains nightly and auto-deploys, so the day the incoming data is bad or mislabeled, a worse (or broken) model ships straight to users automatically. Always gate promotion on "beats the current Production model on the held-out set," and validate the incoming data first (8.1). Automation amplifies whatever you point it at, including mistakes.

WHEN NOT TO DO CONTINUOUS TRAINING

If your data and the input→output relationship are stable, scheduled retraining is wasted compute and added risk — retrain **on drift** (Part 7) or on a slow cadence instead of nightly. And a low-stakes or rarely-used model may not need automated CT at all; a manual retrain-and-promote when metrics slip is fine. Trigger retraining by need, not by reflex.

EXERCISE 8.1

Build a CI check that, on new data, validates the data schema, trains a model, evaluates it on a fixed held-out set, and **fails** unless it beats the current Production model — promoting to Staging only on pass. Then wire a trigger (schedule or a drift signal from Part 7) to run it, and confirm a deliberately worse candidate is rejected rather than promoted.

Part 9 · Feature Stores (Advanced)

A feature store is the heaviest piece of MLOps machinery, and the one you're most likely *not* to need — so this chapter is as much about recognising when it's overkill as how it works. It solves a specific, real problem that appears at scale: computing the same features consistently for training and serving, across many models and teams.

9.1 The problem it solves

Two problems, really, both from earlier chapters made worse by scale:

- **Training/serving skew** (Part 6) — features must be computed identically for training (batch, historical) and serving (real-time, per request). At scale, keeping two code paths in sync fails.
- **Duplication** — many models reuse the same features ("customer 30-day spend"). Without a shared store, every team recomputes them, inconsistently.

A feature store centralises feature definitions and serves them both **offline** (for training, point-in-time correct) and **online** (low-latency, for serving) from one definition.

DIAGRAM

```

feature definition (written once)
├── offline store → training data (historical, point-in-time correct)
└── online store  → serving (fast lookups per request)
same definition → no train/serve skew, no duplication
  
```

9.2 Point-in-time correctness

The subtle, critical feature: for training you need each feature's value **as it was at the time of the event**, not today's value — otherwise you leak future information into training and get great offline metrics that collapse in production. Feature stores handle these point-in-time joins; hand-rolling them correctly is genuinely hard.

ENGINEER'S MENTAL MODEL

A feature store is a specialized cache + database with two faces — a columnar/warehouse face for training and a key-value face for low-latency serving — fed by one definition so both agree. Tools: **Feast** (open source), or the managed stores in SageMaker/Vertex/Databricks. Think of it as the single source of truth for "what does feature X mean and what's its value," the way a shared library is the source of truth for a function.

9.3 Be honest: you probably don't need one yet

A feature store is infrastructure with real operational cost. It pays off with **many models sharing features, a team, and real-time serving needs**. For a single model, or batch-only serving, a shared preprocessing module (used by both training and serving) solves training/serving skew far more cheaply.

WHEN NOT TO BUILD A FEATURE STORE

Don't stand one up for one or a few models, batch-only serving, or a solo project — it's heavy infrastructure whose benefits (sharing, online/offline consistency at scale, point-in-time joins across many features) simply aren't in play. A well-shared preprocessing function plus your registry and monitoring covers the same ground for small setups. Reach for a feature store when *feature sharing and real-time consistency across many models* is the actual bottleneck.

COMMON MISTAKE

Adopting a feature store because it's on the "real MLOps" checklist, then maintaining complex infrastructure to serve two models that don't share features. You've taken on the cost without the benefit. Let the pain justify the tool: build it when duplicated/inconsistent features across models is a problem you can *point to*, not preemptively.

EXERCISE 9.1

First, solve training/serving skew the *cheap* way: extract feature computation into one module imported by both training and serving, and confirm both produce identical features. Then read Feast's quickstart and define one feature with an offline and online store, doing a point-in-time-correct training join — experiencing what the machinery buys you, and deciding whether your case justifies it.

Part 10 · Capstone & Cheat Sheet

10.1 Capstone — take a model from notebook to operated service

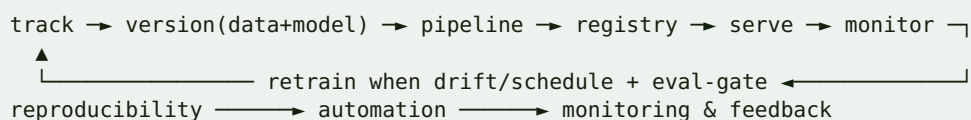
Wrap a model you built earlier in the full (right-sized) MLOps loop:

1. **Track** every training run with MLflow — params, metrics, artifact, data version, git SHA (Part 2).
2. **Version** the data (DVC) and register the model, so any version is reproducible (Parts 3, 5).
3. **Pipeline** the training as parameterised, one-command steps (Part 4).
4. **Register & promote** the best model through Staging → Production (Part 5).
5. **Serve** the Production model by name+stage behind a container, sharing preprocessing code with training (Part 6).
6. **Monitor** — log every prediction; watch input/prediction drift and (delayed) accuracy (Part 7).
7. **Automate** — CI tests data + model quality; retrain on drift/schedule; promote only if it beats Production (Part 8).

Right-size it: a real but modest model needs Parts 2-7; skip the feature store unless Part 9's problem is actually yours.

10.2 The MLOps loop at a glance

DIAGRAM



10.3 Tooling cheat sheet

DIAGRAM

Experiment tracking	MLflow · Weights & Biases
Data versioning	DVC · LakeFS · versioned object storage
Model registry	MLflow Registry · SageMaker/Vertex registries
Pipelines	script/Makefile · DVC · Airflow/Dagster/Prefect · KubeFlow
Serving	FastAPI+Docker · Triton/TorchServe · vLLM (LLMs) · KServe
Monitoring/drift	Evidently · Prometheus+Grafana · cloud model monitors
Feature store	Feast · SageMaker/Vertex/Databricks (only if you need it)

10.4 The five rules

1. **A model is code + data + a decaying artifact.** Reproduce all of it, and watch it after deploy.
2. **Build left to right:** reproducibility (track + version) before automation before a fancy serving stack.
3. **Deploy by promotion, not by overwriting a file.** Registry + stages give you lineage and one-click rollback.
4. **Share preprocessing between train and serve.** Training/serving skew is the silent killer.
5. **Monitor quality, not just uptime, and gate every promotion** on beating the current Production model.

YOU'RE READY

You can now take a model from a notebook to a tracked, versioned, pipelined, registered, served, and monitored production system — with automated, gated retraining when it drifts. This is the engineering half of the ML engineer role, and it's the half most people are missing. Right-size the rigor to the stakes, build the foundation before the flourishes, and never let a model run unwatched. Now go operate one.